

Test Plan

1. Project Scope

1.1 Background

This test plan is written for the **independent application under test**, not for the AutoTestDesign tool itself.

The application under test is **MiniShop Checkout**, a compact e-commerce checkout prototype implemented in:

- `target_app/minishop_checkout/`

Its formal definition is documented in:

- `final_docs/12_target_application_definition_cn.md`

The final project uses **ARG-Test** as the AutoTestDesign tool to test **MiniShop Checkout**. The tool is responsible for:

- requirement ingestion and structuring
- risk-aware prioritization
- systematic black-box test generation
- export of reviewable artifacts

However, the **system under test in this plan** is **MiniShop Checkout**, not **ARG-Test**.

1.2 Objectives

The objectives of this test plan are:

1. identify the major functional and non-functional test items of **MiniShop Checkout**
2. prioritize testing effort according to application risk
3. choose suitable test techniques for each application component
4. execute one selected major module in detail with black-box and white-box evidence
5. use **ARG-Test** to make the design process more systematic, traceable, and reviewable

1.3 In-Scope and Out-of-Scope Content

In scope:

- coupon and promotion behavior
- shipping fee calculation
- tax and order-total calculation
- payment-card validation
- pickup-station and recipient validation
- checkout orchestration across the above modules
- detailed executable testing for `coupon_discount_engine`
- traceable test artifacts and risk prioritization produced with **ARG-Test**

Out of scope:

- order-history and post-purchase account features
- refund execution and return approval workflows
- inventory synchronization and warehouse dispatch
- external payment gateway integration
- production deployment and load testing of a live web service

The project therefore combines **concrete application testing** for **MiniShop Checkout** with **tool-supported requirement-driven test design** from **ARG-Test**.

2. Test Items

2.1 Major Functional Features

Feature area	Main behavior under test	Main code path	Requirement basis
Promotion and pricing	coupon thresholds, coupon validity, member restriction, sale-item restriction, free-shipping effect	target_app/minishop_checkout/promotion.py, reference_impl/coupon_discount_engine.py	coupon_discount_engine
Shipping calculation	zone-based fee selection, weight tiers, express surcharge, free-shipping threshold	target_app/minishop_checkout/shipping.py	shipping_fee_calculator
Tax and total calculation	taxable base selection, country-specific rate, rounding, final total	target_app/minishop_checkout/tax.py	order_total_tax_calculation
Payment validation	cardholder name, card number, expiry window, CVV length, masked-number rejection	target_app/minishop_checkout/payment.py	payment_card_expiry_and_cvv_validation
Pickup validation	pickup station ID, recipient phone, contactless pickup	target_app/minishop_checkout/pickup.py	pickup_station_contact_validation

Feature area	Main behavior under test	Main code path	Requirement basis
	code, note length		
Checkout orchestration	subtotal, promotion, shipping, tax, and validation signals combined into one preview result	<code>target_app/minishop_checkout/checkout.py</code>	integration of the above requirement concerns

2.2 Major Non-Functional Features

NFR area	Test interpretation for MiniShop Checkout	Planned evidence
Performance	checkout preview and validation should complete quickly for classroom-scale order inputs	local smoke execution of <code>MiniShopCheckoutService</code>
Usability / understandability	order preview outputs and validation messages should be readable enough for review and debugging	<code>target_app/minishop_checkout/README.md</code> , smoke scenarios
Security / data handling	malformed or masked primary card input must be rejected and local artifacts should stay repository-controlled	<code>target_app/minishop_checkout/payment.py</code> , repository-local runs
Maintainability and technology	application modules should stay small, separated by responsibility, and easy to test with <code>pytest</code>	<code>target_app/minishop_checkout/</code> , <code>tests/test_minishop_checkout_smoke.py</code>

2.3 Target Application Architecture and Main Components

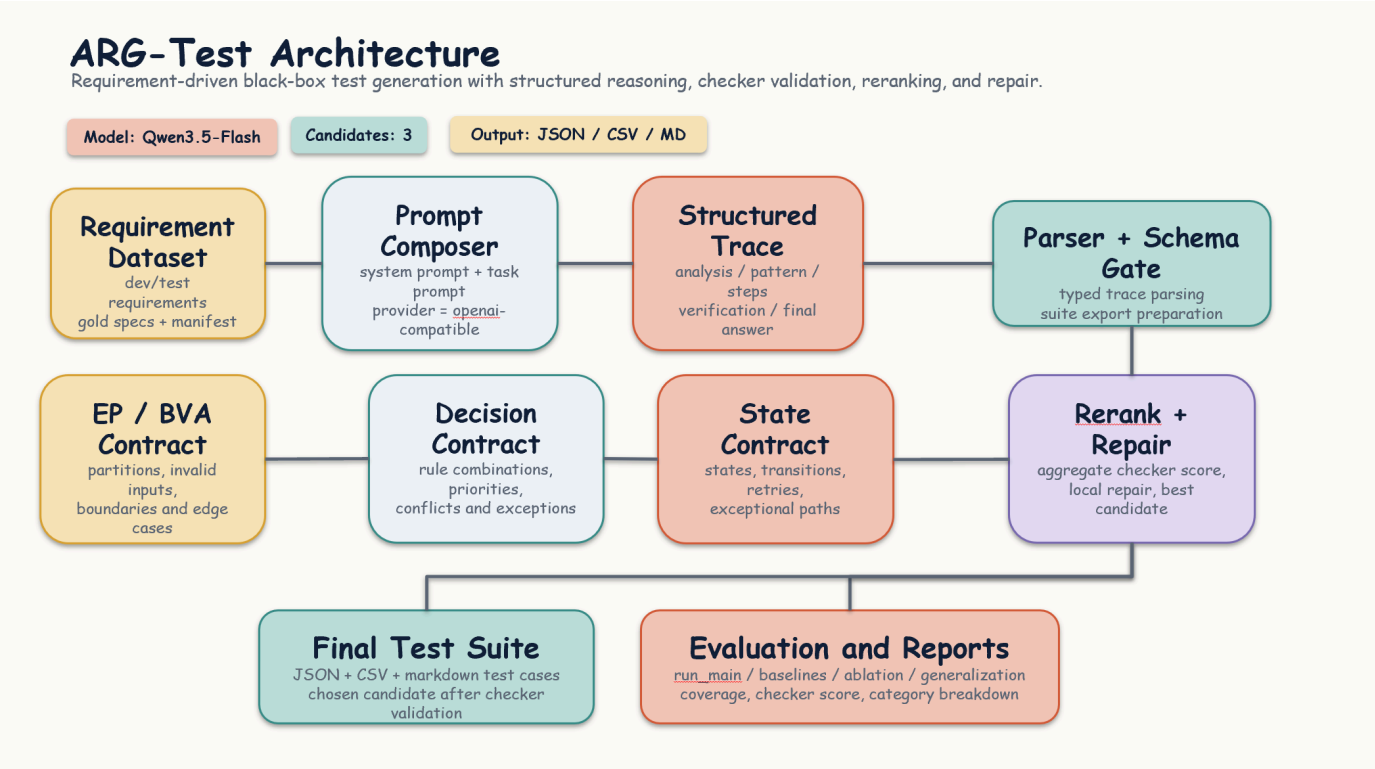
The target application is a compact checkout-oriented system with the following components.

Layer / component	Responsibility	Main code path
Client input layer	collects order, coupon, shipping, payment, and pickup inputs	conceptual application boundary
Checkout service	coordinates subtotal, promotion, shipping, tax, and validation	<code>target_app/minishop_checkout/checkout.py</code>
Promotion service	coupon validation and discount application	<code>target_app/minishop_checkout/promotion.py</code> , <code>reference_impl/coupon_discount_engine.py</code>
Shipping service	shipping fee calculation	<code>target_app/minishop_checkout/shipping.py</code>
Tax service	tax and total calculation	<code>target_app/minishop_checkout/tax.py</code>
Payment validation service	payment-card validation	<code>target_app/minishop_checkout/payment.py</code>

Layer / component	Responsibility	Main code path
Pickup validation service	pickup contact validation	target_app/minishop_checkout/pickup.py

2.4 Supporting AutoTestDesign Tool Path

MiniShop Checkout is tested with ARG-Test, whose own internal support architecture is shown below for traceability of the testing workflow.



The figure above explains how MiniShop Checkout requirements are ingested, structured, checked, reranked, repaired, and exported. It is included here as the **supporting test-tool path**, not as the architecture of the system under test.

3. High-Level Test Suite Design

3.1 Planned Test Suites

Suite ID	Suite name	Main application focus	Main techniques	Primary goal
A	Promotion suite	coupon validity, threshold edges, member restrictions, sale-item conflict	Decision Table + EP + BVA	cover business-rule combinations and discount thresholds
B	Shipping and tax suite	weight tiers, zone tiers, free-shipping threshold, tax-rate and rounding behavior	BVA + Decision Table	cover mathematical correctness and threshold edges
C	Payment validation suite	required fields, format, length, range, and masked-card rejection	EP + BVA	cover valid/invalid payment partitions
D	Pickup validation suite	pickup station format, phone format, contactless pickup code rules	EP + BVA	cover required/optional and valid/invalid partitions

Suite ID	Suite name	Main application focus	Main techniques	Primary goal
E	Checkout orchestration suite	integration between subtotal, promotion, shipping, tax, payment, and pickup outputs	scenario-based integration checks	verify consistency of the overall checkout preview
F	Detailed executable module suite	<code>coupon_discount_engine</code>	EP + BVA + Decision Table + white-box branch testing	connect requirement-driven design to executable evidence
G	NFR and traceability suite	maintainability, basic performance, reviewability, reproducibility	smoke execution, artifact verification	ensure the target-application package remains explainable and rerunnable

3.2 Technique Selection by Risk

Risk profile	Technique choice	Rationale
Coupon and pricing rules	Decision Table + BVA + EP	needed for interacting conditions and threshold edges
Shipping and tax calculation	BVA + Decision Table	needed for tier selection, country-rate choice, and rounding
Payment and pickup validation	EP + BVA	best for valid/invalid partitions and inclusive boundaries
Checkout orchestration	scenario-based integration checks	needed to verify that module outputs remain consistent when combined
Selected executable module	EP + BVA + Decision Table + white-box	satisfies both black-box and white-box expectations for the detailed document

3.3 Designer-in-the-Loop Review Strategy

The assignment requires designer participation and interactive review. In this project, the testing workflow is therefore organized as:

1. select or enter a `MiniShop Checkout` requirement
2. let `ARG-Test` generate a structured trace and candidate suite
3. review risk signals, selected techniques, generated cases, and diagnostics
4. revise requirement wording or designer guidance when needed
5. export the reviewed suite and use the detailed module for executable validation where applicable

The current practical review surfaces are:

- Direct Input
- CSV Batch
- State Model
- Formal Evidence

The current interactive review controls support:

- designer-selected technique emphasis
- explicit coverage-item review notes
- rerun-based revision of generated suites with preserved review metadata

3.4 Evidence Chain and Traceability

Plan item	Application concern	Verification activity	Main evidence
Promotion-rule coverage	coupon validity, threshold, and exclusivity logic	inspect generated obligations and detailed module evidence	<code>outputs/final_tests/</code> , <code>final_docs/execution_evidence/</code>
Shipping/tax correctness	wrong fee, taxable base, or rounding	review generated cases and smoke calculations	<code>target_app/minishop_checkout/</code> , smoke scenarios
Payment validation coverage	malformed payment data accepted or rejected incorrectly	inspect valid/invalid partitions and boundary cases	generated suites, <code>target_app/minishop_checkout/payment.py</code>
Pickup validation coverage	bad pickup contact data accepted	inspect EP/BVA cases and smoke checks	generated suites, <code>target_app/minishop_checkout/pickup.py</code>
Checkout integration consistency	incorrect total or dropped validation signals	execute end-to-end preview scenarios	<code>target_app/minishop_checkout/checkout.py</code> , <code>tests/test_minishop_checkout_smoke.py</code>
Detailed executable module	one major module must be executed in detail	run <code>pytest</code> , <code>coverage.py</code> , and mutation evidence for <code>coupon_discount_engine</code>	<code>tests/</code> , <code>reference_impl/</code> , <code>final_docs/execution_evidence/</code>

4. Test Levels and Goals

Test level	Goal	Typical output
Requirement-level design	verify that <code>MiniShop Checkout</code> requirements are translated into correct coverage items and techniques	structured traces, risk scores, generated suites
Component-level execution	verify shipping, tax, payment, pickup, and orchestration behavior with executable smoke scenarios	smoke outputs, component assertions
Module-level detailed execution	verify one selected major application module with full executable evidence	<code>pytest</code> results, coverage, mutation evidence
Acceptance / submission level	verify that all deliverables consistently describe <code>MiniShop Checkout</code> and its selected module	final documents, evidence package, PPT wording

5. Schedule and Checklist

The schedule below is organized around **testing `MiniShop Checkout`**, not around generic repository maintenance.

Phase	Main activity	Deliverable / checkpoint
Phase 1	freeze the target-application definition and module map	<code>12_target_application_definition_cn.md</code> , <code>target_app/minishop_checkout/</code>
Phase 2	perform application risk analysis and prioritize suites	risk register and suite priorities
Phase 3	generate and review application-facing suites with <code>ARG-Test</code>	reviewed black-box suites for the main components

Phase	Main activity	Deliverable / checkpoint
Phase 4	perform detailed executable testing for <code>coupon_discount_engine</code>	black-box tests, white-box tests, coverage, mutation evidence
Phase 5	run target-application smoke checks and verify traceability	smoke scenarios, consistent code/document references
Phase 6	consolidate documents and submission artifacts	PDF documents, PPT/PDF, demo package, zipped scripts

Pre-submission checklist:

1. the risk report, test plan, and detailed execution document all describe `MiniShop Checkout`
2. `coupon_discount_engine` is consistently described as a selected module of `MiniShop Checkout`
3. the main risk areas have assigned suites and stated techniques
4. the selected executable module has black-box and white-box evidence
5. the final package does not confuse `ARG-Test` with the application under test

6. Organization Structure and Responsibilities

The team structure below describes responsibilities for testing `MiniShop Checkout` with `ARG-Test`.

Member	Student ID	Main responsibility in target-application testing
Yi Xu	2351441	overall integration, application-scope consistency, tool-run control, final merge
Luowu Zhang	2352746	document integration, PPT, demo assets, wording consistency across application-facing deliverables
Xiang Wang	2351039	requirement assets, target-application scenario maintenance, traceability support
Fengxuan Kang	2350283	selected executable module support, <code>pytest</code> execution support, smoke checks
Yiwei Chen	2350217	evaluation interpretation, reproducibility support, verification materials

Responsibility flow:

- freeze application scope first
- align risk and suite priorities next
- review generated suites and adjust wording if needed
- execute the selected detailed module
- synchronize documents and evidence to the same target-application wording

7. Chosen Testing Framework and Rationale

7.1 Main Execution Framework

`pytest` is the chosen execution framework for the executable parts of `MiniShop Checkout`, especially the selected detailed module `coupon_discount_engine`.

Why `pytest`:

- the target application prototype is implemented in Python
- black-box, white-box, and smoke scenarios can all be expressed directly as assertions
- the framework integrates naturally with `coverage.py`
- it supports stable local reruns and selective execution for the final evidence package

7.2 Supporting Tools

Tool	Role in testing MiniShop Checkout
ARG-Test	generate structured requirement-driven suites and risk ordering
pytest	execute detailed black-box, white-box, and smoke tests
coverage.py	collect statement and branch coverage for the selected detailed module
mutation demo scripts	show defect-detection usefulness of the selected detailed module suite

This means **ARG-Test** is the **design-support tool**, while **pytest** is the **execution framework** used on the target application.

8. Cost Estimation

This cost estimate is for **using the developed AutoTestDesign tool to test MiniShop Checkout**. It does **not** count the original R&D cost of building **ARG-Test** itself.

8.1 Estimated Workload with AutoTestDesign

Work item	Estimated workload
normalize MiniShop Checkout requirement sources and component map	0.5 to 1.0 person-day
risk analysis and suite prioritization	0.5 to 1.0 person-day
automated generation plus human review of component-facing suites	1.5 to 2.0 person-days
selected module executable testing with pytest and coverage.py	1.5 to 2.0 person-days
result consolidation and traceability packaging	0.5 to 1.0 person-day

Estimated total with the tool:

- **4.5 to 7.0 person-days**

8.2 Manual-Testing Baseline Estimate

If the same target-application scope were tested manually without **ARG-Test**, the expected workload would increase because the team would have to:

- manually decompose each requirement into coverage items
- manually select techniques for each component
- manually keep risk priority and traceability aligned
- manually format and maintain export artifacts

Estimated manual workload:

- **7.5 to 10.0 person-days**

8.3 Interpretation

The main cost reduction from **ARG-Test** comes from:

- faster requirement decomposition
- faster first-pass suite generation
- faster risk-based ordering
- lower traceability and artifact-formatting overhead

Human review is still required for:

- checking semantic correctness of generated cases
- confirming high-risk rule combinations
- selecting and executing the detailed module

- packaging final evidence for submission and Q&A
-

9. Exit Criteria

This test plan reaches its delivery condition when all of the following are true:

1. **MiniShop Checkout** has a fixed scope and component definition
2. the risk analysis report, test plan, and detailed execution document all point to the same application under test
3. high-risk application areas have mapped suites and selected techniques
4. **coupon_discount_engine** has complete black-box and white-box execution evidence
5. the target-application smoke scenarios are locatable and explainable
6. the final package does not confuse **ARG-Test** with **MiniShop Checkout**

For the selected module, the concrete execution acceptance target is:

- 15 module-focused tests passed
 - 100% statement coverage
 - 100% branch coverage
 - 4 / 4 mutants killed
-

10. Conclusion

This test plan is centered on **MiniShop Checkout** and uses **ARG-Test** as the testing tool that supports requirement-driven design, prioritization, and traceability.

Its most important conceptual clarification is:

- **MiniShop Checkout** is the application under test
- **ARG-Test** is the tool used to test it
- **coupon_discount_engine** is the selected major module of **MiniShop Checkout** used for detailed execution

With that separation fixed, the plan aligns with the assignment requirement that the test plan must describe how the team tests the **target application**, not the AutoTestDesign tool itself.